МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»»

Институт компьютерных наук

Кафедра инженерной кибернетики

ОТЧЕТ

по практической работе №2

по дисциплине «Современные интеллектуальные сетевые сервисы»

Развитие AI-сервиса: сквозная идентификация запросов, асинхронная обработка и интеграция LLM

Выполнил:
студент группы МПИ-25-1-2
Варецкий Б. В.

Проверил:
М. Е. Корнет

Оценка: _____

Москва, 2026

Содержание

1 Цель работы **2**

2 Архитектура расширенного AI-сервиса **2**

3 Исходный код и среда выполнения **3**

4 Выполнение заданий **4**
4.1 Задания 1 и 2. Сквозной request_id и middleware 4
4.2 Задание 3. Обработка ошибок API 6
4.3 Задание 4. SQLite через aiosqlite 7
4.4 Задание 5. Сохранение результата /predict 8
4.5 Задание 6. Чтение прогнозов из БД 10
4.6 Задание 7. Пагинация по дню недели 10
4.7 Задание 8. Явный пакетный инференс 12
4.8 Задание 9. Очередь и dynamic batching 15
4.9 Задание 10. Backpressure и /queue-status 16
4.10 Задание 11. Таймаут инференса 17
4.11 Задания 12, 13 и 14. Подключение внешней LLM 18

5 Тестирование API (задание 15) **20**

6 API как контракт (задание 16) **21**

7 Путь одного запроса через систему **21**

8 Выводы **22**

1 Цель работы

Цель работы: развить REST API сервиса агроскоринга, созданный в практической работе №1, и приблизить его архитектуру к промышленному AI-сервису.

В сервис добавлены механизмы, которые нужны при работе под нагрузкой: сквозная идентификация запросов (`request_id`) через промежуточный слой (middleware); одиночный, явный пакетный и динамический пакетный инференс с очередью `asyncio.Queue`; ограничение очереди (backpressure) и таймауты; хранение результатов в SQLite через `aiosqlite`; асинхронное обращение к внешней LLM через `httpx.AsyncClient`; обработка ошибок внешних зависимостей.

2 Архитектура расширенного AI-сервиса

Сервис реализован в одном модуле `main.py` на FastAPI. Последовательность обработки запроса показана на рисунке 1.

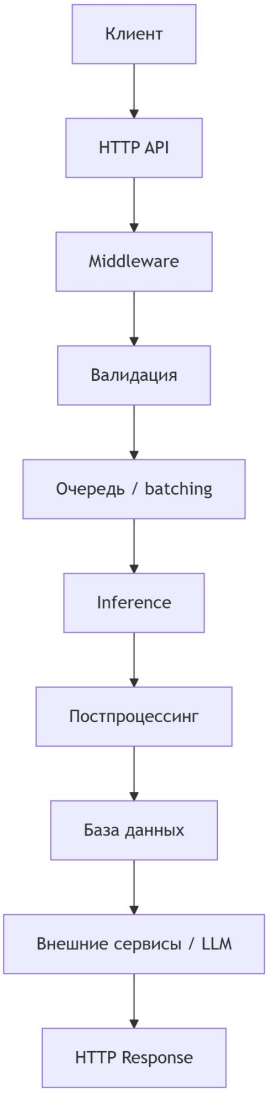


Рис. 1. Обработка запроса в API AI-сервиса агроскоринга

Назначение компонентов:

- Middleware `request_context_middleware` для каждого HTTP-запроса берет `X-Request-ID` из заголовка или генерирует UUID4, сохраняет его в `request.state.request_id`, пишет в лог начало и конец запроса и добавляет в ответ заголовки `X-Request-ID` и `X-Process-Time`. Необработанное исключение middleware преобразует в ответ 500 с `request_id`.
- Валидация выполняется в два этапа. Pydantic-схемы (`FarmRequest`, `BatchRequest`, `LLMChatRequest`) проверяют типы и диапазоны значений, при ошибке возвращается 422. Функция `validate_business_rules` проверяет правила предметной области, например допустимый регион, при ошибке возвращается 400.
- Инференс доступен в трех режимах: одиночный `/predict`, явный пакетный `/batch-predict` и динамический пакетный `/predict-queued`. Модель имитируется функцией `calculate_risk`, время вызова $T(n) = 20 + 3n$ мс. Объект `asyncio.Lock` имитирует единственный экземпляр модели, который обрабатывает вызовы по одному.
- Очередь `asyncio.Queue(maxsize=100)` обслуживает фоновая корутина `batch_worker`. Она собирает динамический пакет и через `asyncio.Future` передает каждому запросу его результат.
- Результаты хранятся в SQLite в режиме WAL, доступ выполняется через `aiosqlite`. Каждая операция открывает отдельное соединение с `busy_timeout = 5000` мс.
- Внешняя LLM подключена через OpenAI-совместимый endpoint Vireonix. Для всего приложения используется один `httpx.AsyncClient`, который создается и закрывается в `lifespan`.
- Исключения `HTTPException` и `RequestValidationError` обрабатываются глобально, тело ответа с ошибкой содержит поля `detail` и `request_id`.

Эксплуатационные ограничения заданы константами (таблица 1).

Таблица 1. Эксплуатационные параметры сервиса

Параметр	**Значение**	**Назначение**
`MAX_BATCH_SIZE`	8	максимальный размер пакета (явного и динамического)
`MAX_BATCH_WAIT_MS`	50	окно накопления динамического пакета, мс
`QUEUE_MAX_SIZE`	100	предел очереди, при переполнении возвращается 429
`INFERENCE_TIMEOUT_SECONDS`	2.0	таймаут ожидания результата модели, с
`MODEL_FIXED_OVERHEAD_SECONDS`	0.020	фиксированные накладные расходы вызова модели
`MODEL_PER_ITEM_SECONDS`	0.003	время обработки одного объекта
`DB_BUSY_TIMEOUT_MS`	5000	ожидание блокировки SQLite, мс
`LLM_TIMEOUT_SECONDS`	60.0	таймаут HTTP-запроса к LLM, с
`LLM_DEFAULT_MAX_TOKENS`	300	ограничение длины ответа LLM
`LLM_DEFAULT_TEMPERATURE`	0.3	температура генерации

3 Исходный код и среда выполнения

Исходный код: ссылка на репозиторий.

Запуск сервиса:

```
pip install fastapi uvicorn aiosqlite httpx
uvicorn main:app --host 127.0.0.1 --port 8000
# Swagger UI: http://127.0.0.1:8000/docs
```

Листинг 1. Установка зависимостей и запуск

Проверка выполнялась в окружении Python 3.13, FastAPI 0.162, Starlette 1.7, Pydantic 2.13, aiosqlite 0.22, httpx 0.28, uvicorn 0.54, Swagger UI 5. Функциональные сценарии проверялись через Swagger UI. Сценарии с параллельной нагрузкой (переполнение очереди, сравнение режимов инференса) и с искусственными отказами (таймаут 0.001 с, недоступная БД, падение модели, ошибка LLM-провайдера) выполнялись скриптом на `httpx`. Для них сервис запускался с измененным значением нужной константы, исходный `main.py` не менялся.

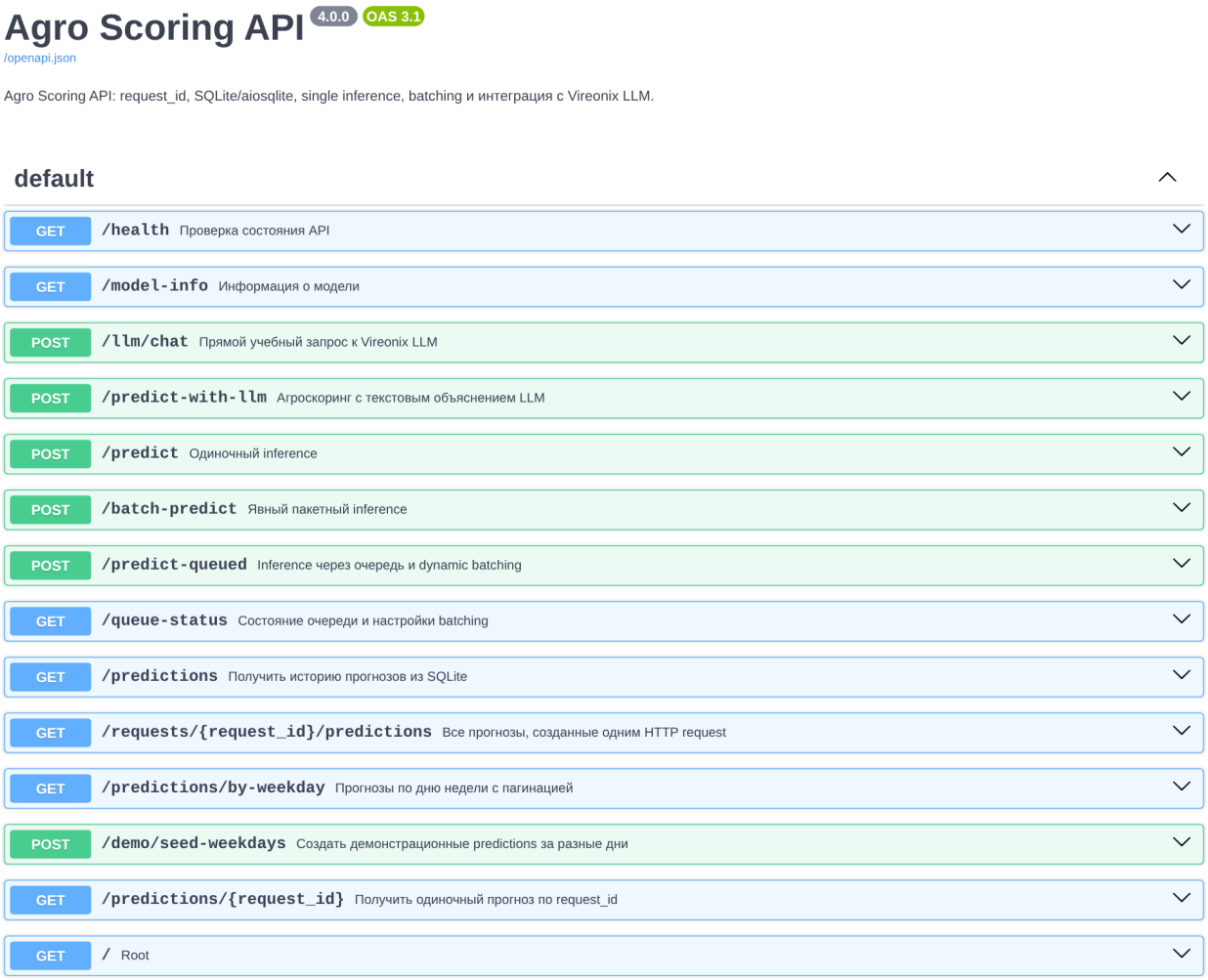


Рис. 2. Swagger UI сервиса: список endpoints

4 Выполнение заданий

4.1 Задания 1 и 2. Сквозной request_id и middleware

Middleware выполняется для каждого HTTP-запроса до вызова обработчика и после него (листинг 2).

```python
@app.middleware("http")
async def request_context_middleware(request: Request, call_next):
    start_time = time.perf_counter()
    request_id = request.headers.get("X-Request-ID")
    if not request_id:
        request_id = str(uuid.uuid4())
    request.state.request_id = request_id
    logger.info("Request started | request_id=%s | method=%s | path=%s",
                request_id, request.method, request.url.path)
    try:
        response = await call_next(request)
    except Exception:
        logger.exception("Unhandled error | request_id=%s", request_id)
        response = JSONResponse(status_code=500, content={
            "detail": "Internal server error", "request_id": request_id
        })
    process_time = time.perf_counter() - start_time
    response.headers["X-Request-ID"] = request_id
    response.headers["X-Process-Time"] = str(round(process_time, 6))
    logger.info("Request completed | request_id=%s | status=%s | time=%.4f",
                request_id, response.status_code, process_time)
    return response
```

Листинг 2. Middleware: request_id и X-Process-Time (сокращено)

Обработчики берут идентификатор из `request.state.request_id` и передают его в ответ, в лог и в запись SQLite. Проверка выполнена в двух вариантах.

В первом варианте в `POST /predict` был передан заголовок `X-Request-ID` со значением `23f3f05c-5a0f-432c-a91f-f8b5011f8840`. Этот идентификатор присутствует:

- в теле ответа: `"request_id": "23f3f05c-5a0f-432c-a91f-f8b5011f8840"`;
- в заголовках ответа: `x-request-id: 23f3f05c-...`, `x-process-time: 0.027702`;
- в логе (листинг 3);
- в таблице `predictions` (строка `id = 2` в листинге 6).

```
2026-10-25 21:22:20,515 | INFO | Request started | request_id=23f3f05c-5a0f
   -432c-a91f-f8b5011f8840 | method=POST | path=/predict
2026-10-25 21:22:20,542 | INFO | Request completed | request_id=23f3f05c-5
   a0f-432c-a91f-f8b5011f8840 | status=200 | time=0.0277
```

Листинг 3. Записи лога для запроса с клиентским X-Request-ID

Во втором варианте заголовок не передавался. Сервер сгенерировал UUID4 и вернул его в теле ответа и в заголовке `x-request-id` (рисунок 3).

Сравнение X-Process-Time

Для `/health`, `/predict` и `/predict-queued` выполнено по 30 последовательных запросов. В таблице 2 приведена медиана заголовка `X-Process-Time`.

Таблица 2. Время обработки запроса по заголовку X-Process-Time

Endpoint	**Медиана, мс**	**Мин., мс**	**Из чего складывается**
`GET /health`	0.35	0.28	только middleware и сериализация ответа
`POST /predict`	28.7	27.5	валидация, модель 23 мс ($20 + 3 \cdot 1$), запись в SQLite
`POST /predict-queued`	79.8	78.4	то же, что в `/predict`, и окно накопления пакета 50 мс
`POST /predict-with-llm`	?	-	`/predict`, сетевой вызов LLM и генерация текста

`/health` не обращается к модели и к БД, поэтому отвечает менее чем за миллисекунду. Время `/predict` в основном определяется вызовом модели (23 мс). Для одиночного запроса в `/predict-queued` добавляется около 50 мс: worker ждет другие запросы, чтобы собрать пакет, и запускает модель только после окончания окна. При низкой нагрузке dynamic batching увеличивает задержку. Время `/predict-with-llm` определяется ответом внешней LLM и значительно больше, чем у остальных endpoints.

4.2 Задание 3. Обработка ошибок API

Ошибки обрабатываются в трех местах:

- `http_exception_handler` обрабатывает все `HTTPException` (400, 404, 409, 429, 502, 503, 504) и сохраняет заголовки исключения, например `Retry-After`;
- `validation_exception_handler` обрабатывает ошибки Pydantic и параметров запроса (422);
- блок `try/except` в middleware перехватывает остальные исключения и возвращает 500.

Все три обработчика берут `request_id` из `request.state`, поэтому он присутствует в каждом ответе с ошибкой. Ошибки внешних зависимостей преобразуются в HTTP-коды в месте возникновения: `aiosqlite.Error` в 503, `asyncio.TimeoutError` в 504, `asyncio.QueueFull` в 429, ошибки `httpx` в 502, 503 или 504.

Каждый код проверен запросом, результаты приведены в таблице 3.

Таблица 3. Проверка обработки ошибок

Код	**Как получен**	**Фактический `detail`**
400	`region = "Moscow"`	`Unknown region: Moscow. Allowed regions: [...]`
400	`GET /predictions?risk_level=critical`	`risk_level must be low, medium or high`
404	неизвестный `request_id`	`Prediction not found`
404	`by-weekday`, `page=99`	`Page 99 does not exist. Total pages: 2`
409	`GET /predictions/{id}` для batch	`This request_id contains multiple predictions. Use /requests/{id}/predictions`
422	`area_ha = -100`	`Input should be greater than 0`
422	batch из 9 объектов	`List should have at most 8 items after validation, not 9`
429	400 одновременных `/predict-queued`	`Inference queue is full. Try again later.`, заголовок `Retry-After: 1`

Код	Как получен	Фактический `detail`
500	`calculate_risk` заменена функцией, которая вызывает `RuntimeError`	`Internal server error`
502	LLM-провайдер (локальная заглушка) отвечает HTTP 500	`LLM provider returned an error (HTTP 500)`
503	после старта путь к БД заменен на несуществующий	`Prediction storage is unavailable` (при записи и при чтении)
503	`MODEL_READY = False`	`Model is unavailable`
503	Vireonix недоступен по сети	`LLM service is unavailable`
504	`INFERENCE_TIMEOUT_SECONDS = 0.001`	`Inference timeout`, `Batch inference timeout`
504	LLM отвечает дольше `LLM_TIMEOUT_SECONDS = 3`	`LLM request timeout`

Пример ответа при недоступной БД (`POST /predict`):

```
HTTP/1.1 503 Service Unavailable
x-request-id: 726c4d86-d59d-4c47-bb24-534bd8193dbb

{"detail": "Prediction storage is unavailable",
 "request_id": "726c4d86-d59d-4c47-bb24-534bd8193dbb"}
```

Листинг 4. Ответ 503 при ошибке хранилища

4.3 Задание 4. SQLite через aiosqlite

При старте приложения `init_db()` переводит базу `agro_scoring.db` в режим WAL, устанавливает `synchronous=NORMAL` и создает таблицу с индексами по `request_id` и `risk_level` (листинг 5). Поле `request_id` не является первичным ключом, так как один batch-запрос создает несколько строк с одинаковым идентификатором.

```
CREATE TABLE IF NOT EXISTS predictions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    request_id TEXT NOT NULL,
    farm_id TEXT NOT NULL,
    risk_score REAL NOT NULL,
    risk_level TEXT NOT NULL,
    recommendation TEXT NOT NULL,
    model_version TEXT NOT NULL,
    source TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_predictions_request_id ON predictions(
   request_id);
CREATE INDEX IF NOT EXISTS idx_predictions_risk_level ON predictions(
   risk_level);
```

Листинг 5. Схема таблицы predictions

Поле `source` хранит способ получения прогноза: `single`, `batch`, `queued`, `single+llm` или `seed`. Все SQL-запросы параметризованы, данные пользователя не подставляются в текст запроса.

Ответ на вопрос: почему блокирующий `sqlite3.execute()` нежелателен внутри асинхронного endpoint? Обработчик, объявленный через `async def`, FastAPI выполняет в потоке цикла событий. Цикл событий один на процесс, и переключение между задачами происходит только в точках `await`. Вызовы `sqlite3.execute()` и `commit()` синхронные: они выполняют дисковый ввод-вывод, а при занятой базе ждут освобождения блокировки (в нашем сервисе до 5 с по `busy_timeout`). Пока такой вызов не завершится, поток цикла событий занят. Последствия:

- остальные запросы, включая `/health`, не обрабатываются;
- фоновый `batch_worker` останавливается, очередь не разбирается;
- `asyncio.wait_for` не может прервать ожидание, поэтому таймаут инференса не срабатывает вовремя;
- задержка всех запросов увеличивается на время самой медленной операции с БД.

`aiosqlite` выполняет вызовы `sqlite3` в отдельном потоке и возвращает awaitable-объект. Во время работы с диском цикл событий продолжает обслуживать другие корутины. Ограничение самой SQLite (в каждый момент пишет только одно соединение) сохраняется, его последствия уменьшают режим WAL и `busy_timeout`.

4.4 Задание 5. Сохранение результата /predict

В `POST /predict` результат сохраняется вызовом `await save_prediction(result, source="single")`, и ответ возвращается только после успешного `COMMIT`. Если запись не удалась, `aiosqlite.Error` преобразуется в 503, и клиент не получает код 200 для несохраненного прогноза. Это проверено запуском с недоступной БД (таблица 3).

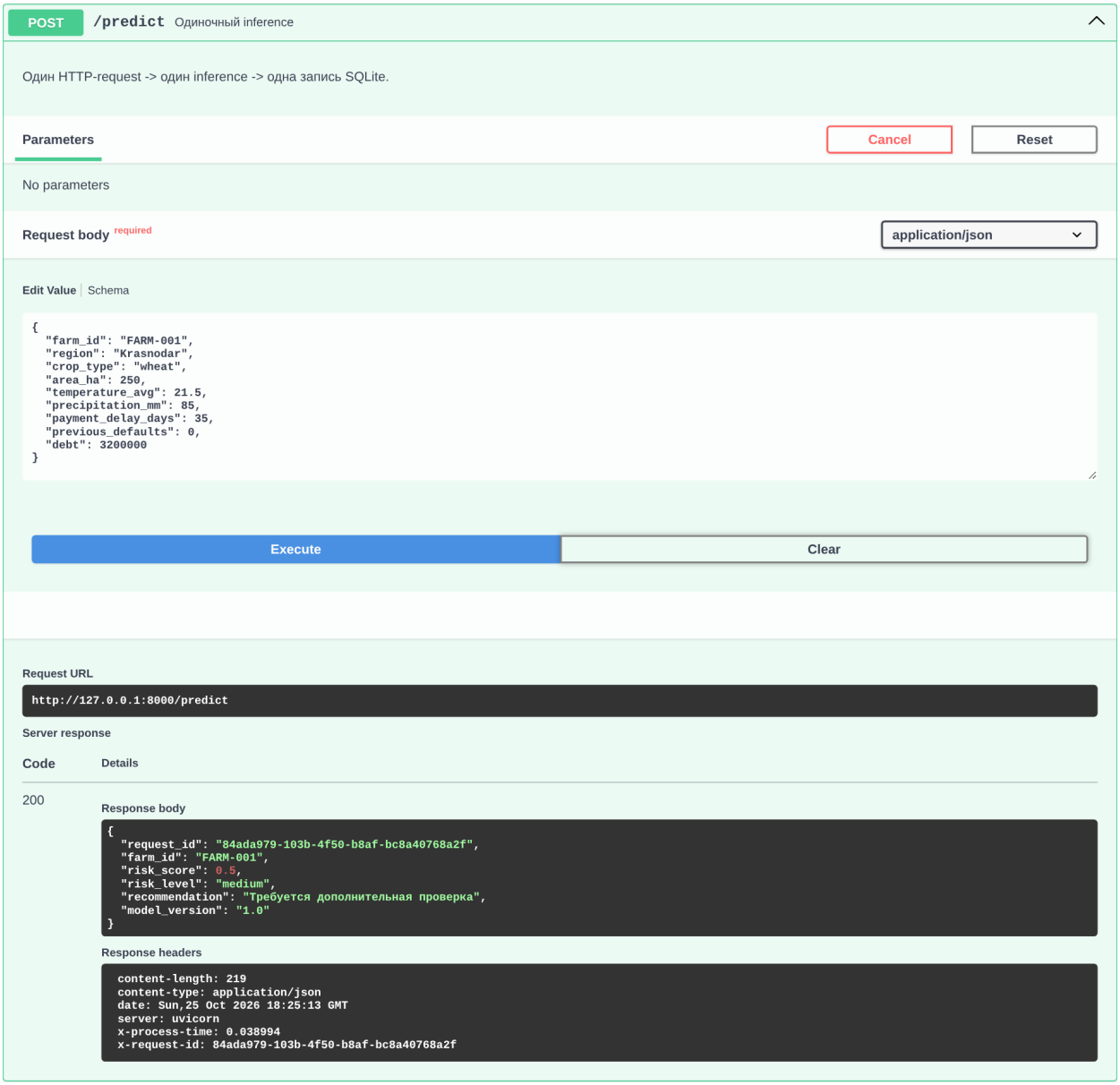


Рис. 3. POST /predict: ответ 200, request_id в теле совпадает с заголовком x-request-id

Записи в базе после выполнения тест-кейсов 2 и 7 (запрос к файлу `agro_scoring.db`):

```
id | request_id   | farm_id   | risk_score | risk_level | source | created_at
---+--------------+-----------+------------+------------+--------+--------------------

1  | ebea24bb...  | FARM-001  | 0.5        | medium     | single | 2026-10-25
   18:22:20
2  | 23f3f05c...  | FARM-002  | 0.5        | medium     | single | 2026-10-25
   18:22:20
3  | dc75c4d3...  | FARM-101  | 0.1        | low        | batch  | 2026-10-25
   18:22:20
4  | dc75c4d3...  | FARM-102  | 0.4        | medium     | batch  | 2026-10-25
   18:22:20
5  | dc75c4d3...  | FARM-103  | 1.0        | high       | batch  | 2026-10-25
   18:22:20
```

Листинг 6. Строки таблицы predictions

4.5 Задание 6. Чтение прогнозов из БД

- `GET /predictions` читает историю из SQLite (`ORDER BY id DESC LIMIT ?`) и поддерживает фильтр `risk_level`. Неверный уровень риска дает 400, значение `limit` вне диапазона $[1, 100]$ дает 422 (рисунок 4).
- `GET /predictions/{request_id}` возвращает запись с кодом 200. При отсутствии записи возвращается 404, а если по идентификатору найдено несколько строк, возвращается 409 с указанием нужного endpoint. Для batch-запроса `dc75c4d3-...` получен 409.
- `GET /requests/{request_id}/predictions` для этого же batch-запроса вернул три прогноза (FARM-101, FARM-102, FARM-103).

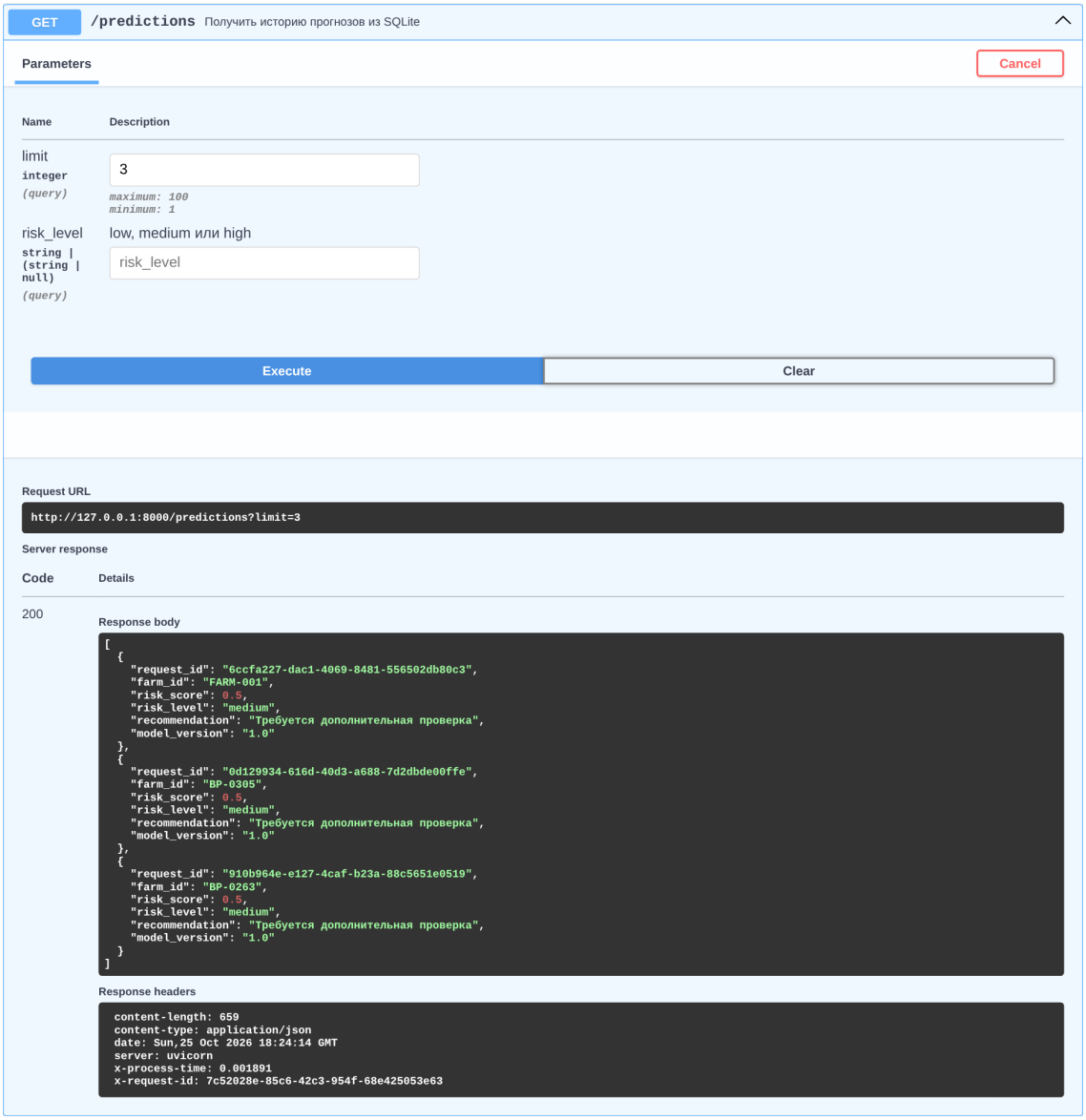


Рис. 4. GET /predictions: последние записи из SQLite

4.6 Задание 7. Пагинация по дню недели

`GET /predictions/by-weekday` использует offset-пагинацию. Записи страницы выбираются запросом с `LIMIT per_page OFFSET (page-1)*per_page`, общее количество записей считается отдельным запросом `COUNT(*)` с тем же условием, число страниц вычисляется как

⌈total_items/per_page⌉. День недели определяется через `strftime('%w', created_at)` и сопоставляется со словарем `WEEKDAY_MAP`. Ссылки `next_page` и `prev_page` формируются из текущего URL.

Для проверки был вызван `POST /demo/seed-weekdays?count=70`. Даты 70 записей распределились по последнему 21 дню, на каждый день недели пришлось по 10 записей. Результаты для понедельника:

- `page=1`: 5 записей, `total_items = 10`, `total_pages = 2`, `next_page` указывает на `page=2`, `prev_page = null` (рисунок 5);
- `page=2`: остальные 5 записей, `next_page = null`, `prev_page` указывает на `page=1`;
- `page=99`: код 404 и сообщение `Page 99 does not exist. Total pages: 2`;
- `page=-1`: 422; `weekday=funday`: 400; запрос без `weekday`: 422, так как параметр обязательный.

Поле `created_at` заполняется через `CURRENT_TIMESTAMP`, то есть по UTC, поэтому день недели тоже считается по UTC. Например, прогноз, сделанный в понедельник в 01:00 по московскому времени, будет отнесен к воскресенью.

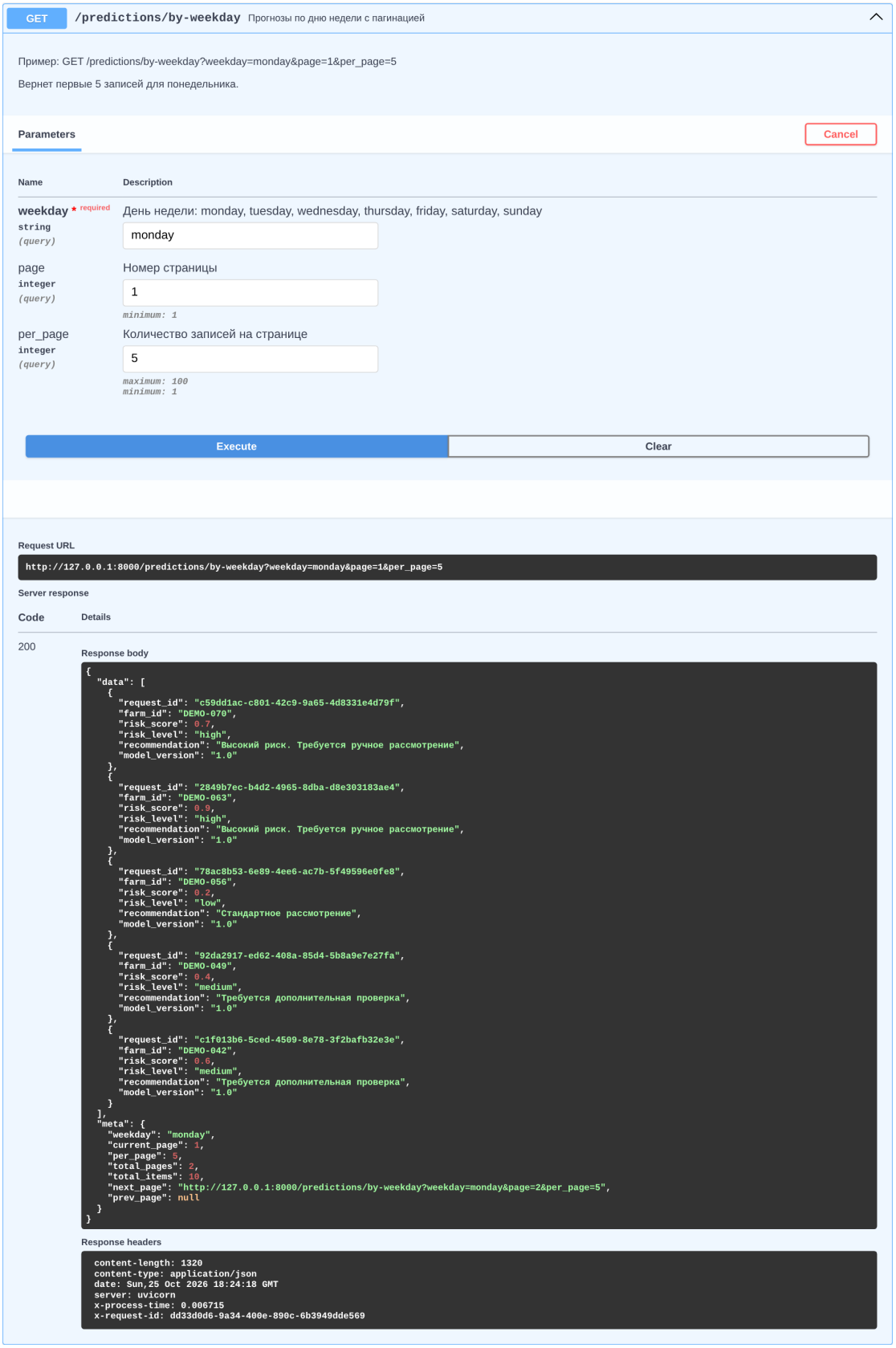


Рис. 5. GET /predictions/by-weekday?weekday=monday&page=1&per_page=5

4.7 Задание 8. Явный пакетный инференс

`POST /batch-predict` принимает `BatchRequest` со списком `items` от 1 до `MAX_BATCH_SIZE = 8` объектов. Ограничение задано через `Field(max_length=...)`, поэтому запрос

с 9 объектами отклоняется с кодом 422 до вызова обработчика. Бизнес-правила проверяются для каждого объекта, затем пакет обрабатывается одним вызовом `model_predict_batch(batch.items)` и сохраняется одной транзакцией в `save_predictions_batch`:

```
async with get_db() as db:
    await db.executemany("INSERT INTO predictions (...) VALUES (?, ?,
       ?, ?, ?, ?, ?);", rows)
    await db.commit()
```

Листинг 7. Сохранение пакета одной транзакцией (сокращено)

Для пакета из трех хозяйств (рисунок 6) получены уровни риска low, medium и high, `processing_time_ms = 33.22`. Из них 29 мс занимает модель ($20 + 3 \cdot 3$), остальное время уходит на запись в БД.

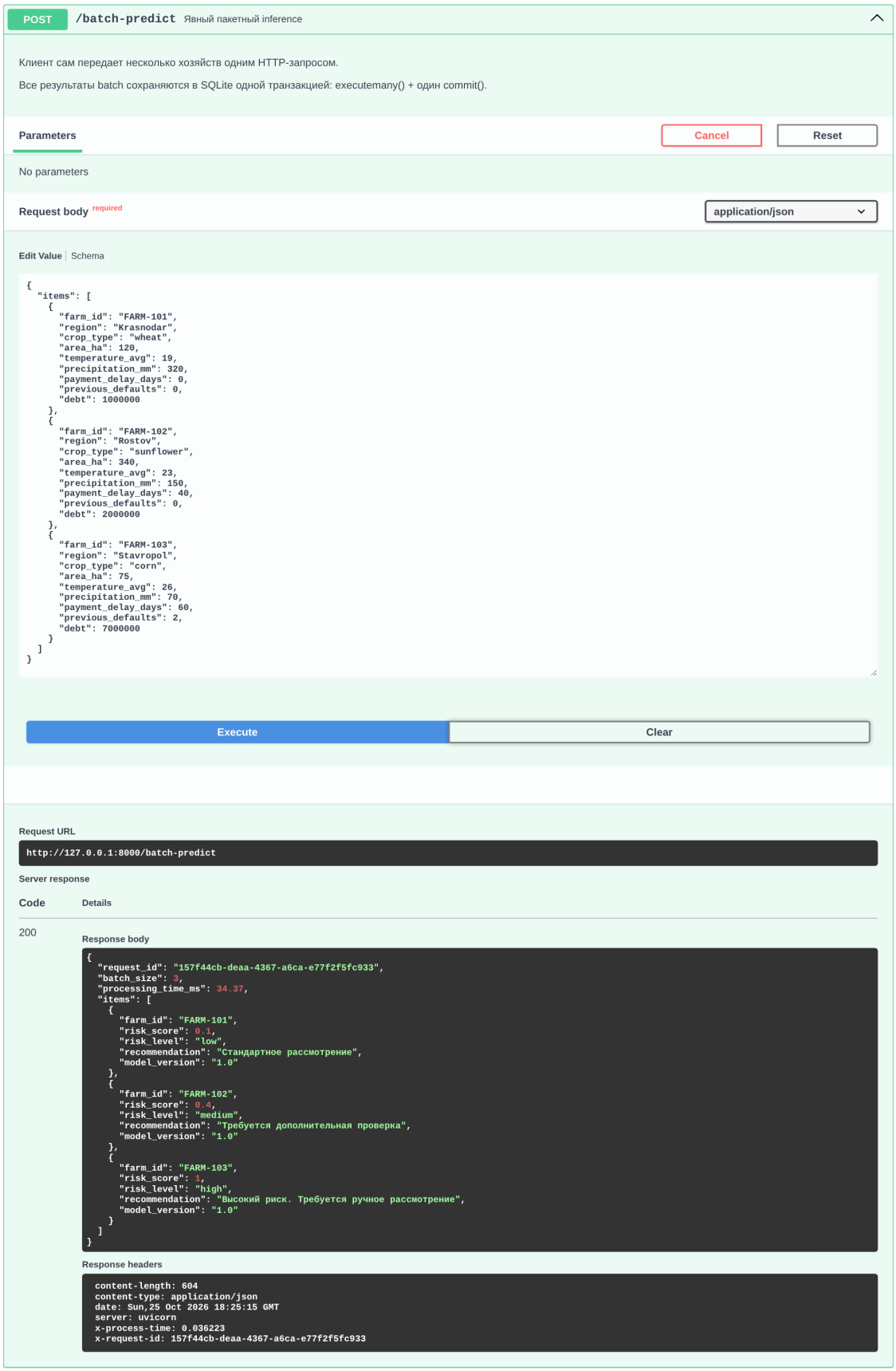


Рис. 6. POST /batch-predict: пакет из трех хозяйств

Ответ на вопрос: почему один `executemany()` предпочтительнее восьми отдельных `INSERT + COMMIT`? Во-первых, пакет сохраняется атомарно: записываются либо все строки, либо ни одной. При восьми отдельных коммитах сбой на пятой записи оставит в ба-

зе 4 прогноза из 8, хотя клиент получит ошибку на весь пакет. Во-вторых, каждый `COMMIT` захватывает блокировку записи и фиксирует изменения в WAL-журнале, а при одном коммите эта работа выполняется один раз. В-третьих, в SQLite в каждый момент пишет только одно соединение, и восемь коротких транзакций чаще заставляют параллельные запросы ждать по `busy_timeout`. Кроме того, каждый `await` в `aiosqlite` передает задачу в поток соединения и обратно, а `executemany` подготавливает выражение один раз и передает все строки за один вызов.

Замер выполнен на той же схеме и с теми же настройками (WAL, `synchronous=NORMAL`), в таблице приведена медиана по 200 повторам для 8 строк:

Таблица 4. Запись 8 прогнозов в SQLite

Способ	**Медиана, мс**
8 × (открыть соединение + INSERT + COMMIT), как 8 вызовов `save_prediction`	23.0
8 × (INSERT + COMMIT) в одном соединении	5.8
`executemany` + один COMMIT, как в `save_predictions_batch`	2.7

Пакетная запись выполняется в 2 раза быстрее, чем восемь коммитов в одном соединении, и примерно в 8 раз быстрее, чем восемь отдельных вызовов `save_prediction`.

4.8 Задание 9. Очередь и dynamic batching

Для клиента `POST /predict-queued` выглядит как обычный одиночный запрос. Обработчик создает `Future`, помещает в очередь `InferenceTask(data, request_id, future)` и ждет этот `Future` не дольше `INFERENCE_TIMEOUT_SECONDS`. Фоновый `batch_worker`, запущенный в `lifespan`, работает по следующему алгоритму:

1. ожидает первый элемент очереди (`await queue.get()`);
2. в течение 50 мс добирает элементы, пока размер пакета не станет равен 8 или не истечет время;
3. пропускает задачи, для которых клиент уже получил таймаут (`future.cancelled()`);
4. вызывает модель один раз для всего пакета;
5. передает результаты через `future.set_result(score)`, i-й результат получает i-я задача. При ошибке модели исключение передается в каждый `Future` пакета.

Обработка одиночного запроса в логе (время ожидания истекло, пакет состоит из одного элемента):

```
21:22:20,611 | Request started   | request_id=7c4a9295-... | path=/predict-
    queued
21:22:20,612 | Request queued    | request_id=7c4a9295-... | queue_size=1
21:22:20,662 | Dynamic batch formed | size=1 | queue_size=0
21:22:20,690 | Request completed | request_id=7c4a9295-... | status=200 |
    time=0.0796
```

Листинг 8. Путь запроса через очередь

При нагрузке пакеты заполнялись полностью. За весь прогон worker сформировал 46 пакетов по 8 объектов, 1 пакет из 4 объектов и 31 пакет из одного объекта. Пакеты из одного объекта получены при последовательных запросах во время замеров `X-Process-Time`.

Сравнение режимов при одновременной отправке 64 объектов приведено в таблице 5.

Таблица 5. Одиночный, динамический и явный пакетный инференс, 64 объекта

Режим	**HTTP-запросов**	**Общее время, с**	**Объектов/с**	**Ср. задержка, мс**
`/predict`	64	1.60	40.0	832
`/predict-queued`	64	0.73	88.2	417
`/batch-predict` (8×8)	8	0.39	163.9	223

Результаты совпадают с расчетом по модели времени. Одиночные вызовы выполняются через `model_lock` по одному: $64 \cdot 23 = 1472$ мс. Восемь пакетов по 8 объектов занимают $8 \cdot (20 + 3 \cdot 8) = 352$ мс, так как фиксированные 20 мс тратятся один раз на пакет. Dynamic batching ускорил обработку в 2.2 раза без изменений на стороне клиента. Явный batching оказался быстрее, потому что состав пакета известен сразу, нет окна ожидания и в БД выполняется одна транзакция вместо восьми.

4.9 Задание 10. Backpressure и /queue-status

Очередь создана с `maxsize = QUEUE_MAX_SIZE = 100`. Перед постановкой задачи endpoint проверяет `queue.full()`, затем вызывает `put_nowait()` и перехватывает `QueueFull`. Если очередь заполнена, клиент сразу получает `429 Too Many Requests` с заголовком `Retry-After: 1`.

Для проверки в `/predict-queued` одновременно отправлено 400 запросов:

- 308 запросов обработаны с кодом 200, 92 получили 429;
- по опросу `/queue-status` размер очереди менялся так: 0, 1, 8, 100, 99, 76, 44, 0, предел 100 не превышался;
- ни один принятый запрос не завершился с кодом 504.

`GET /queue-status` возвращает текущий размер очереди и параметры batching. На рисунке 7 запрос выполнен во время нагрузки, в очереди находилось 84 задачи.

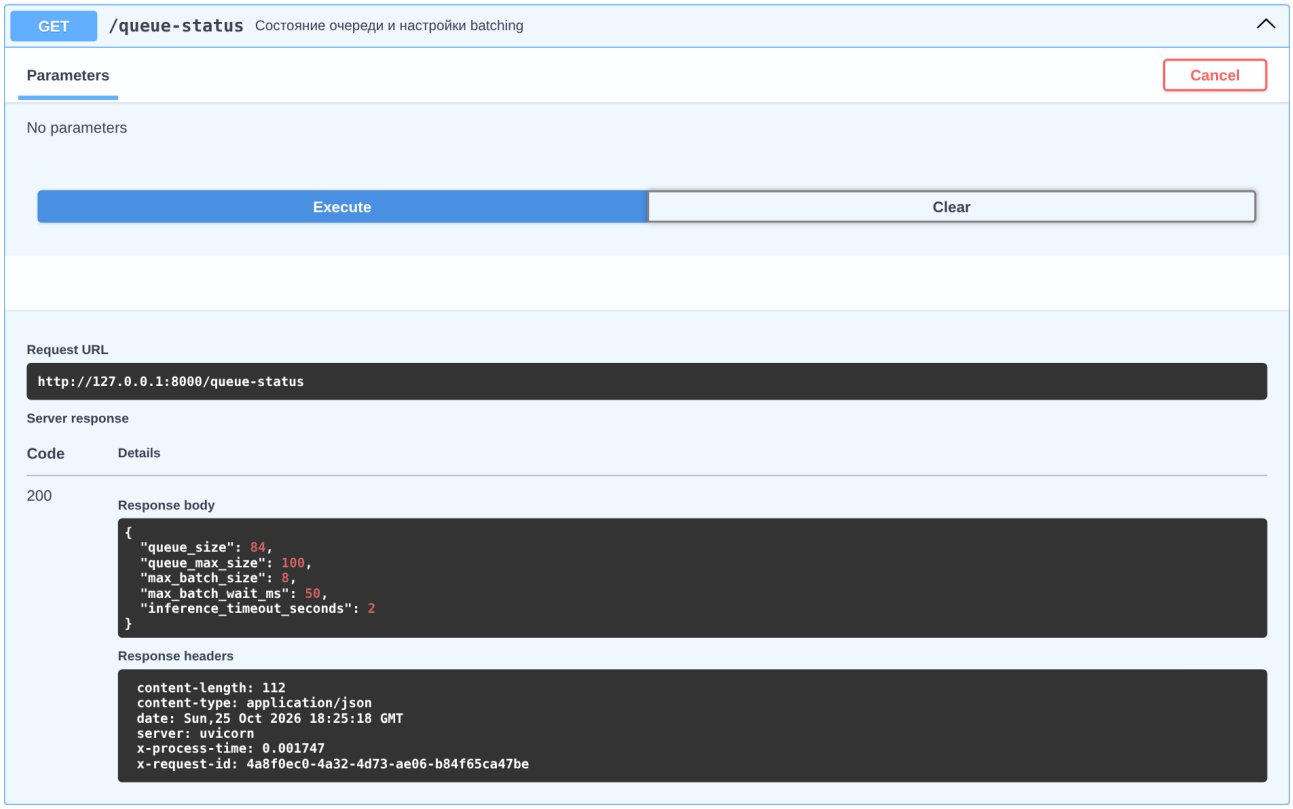


Рис. 7. GET /queue-status под нагрузкой: queue_size = 84 из 100

Ответ на вопрос: что произойдет с памятью и latency, если очередь сделать неограниченной? Если запросы поступают быстрее, чем модель их обрабатывает ($\lambda > \mu$), неограниченная очередь постоянно растет.

Расход памяти увеличивается вместе с длиной очереди, так как каждый ожидающий запрос хранит `InferenceTask` с данными, `Future`, корутину обработчика и открытое соединение клиента. При длительной перегрузке это приводит к нехватке памяти и остановке процесса.

Задержка также растет линейно: время ожидания в очереди примерно равно L/μ, где L обозначает длину очереди. При пакетах по 8 объектов пропускная способность модели составляет около $\mu \approx 8/0.044 \approx 180$ объектов/с. При 1000 задачах в очереди новый запрос ждал бы около 5.5 с, что больше таймаута 2 с. Клиенты получали бы 504 при полностью загруженной модели, а часть вычислений выполнялась бы для запросов, по которым клиент уже получил ошибку. В результате полезная пропускная способность сервиса снижается почти до нуля.

Ограниченная очередь сразу отклоняет лишние запросы кодом 429, и клиент может повторить запрос через время, указанное в `Retry-After`. Время ожидания принятых запросов при этом ограничено: $100/180 \approx 0.55$ с, что меньше таймаута 2 с. Поэтому в эксперименте все 308 принятых запросов завершились успешно.

4.10 Задание 11. Таймаут инференса

Вызов модели во всех режимах выполняется через `asyncio.wait_for(..., timeout=INFERENCE_TIMEOUT_SECONDS)`. В `/predict-queued` с таймаутом ожидается `Future`, после истечения времени он отменяется. Результаты при `INFERENCE_TIMEOUT_SECONDS = 0.001` приведены в таблице 6.

Таблица 6. Результат при INFERENCE_TIMEOUT_SECONDS = 0.001

Запрос	Код	detail	X-Process-Time, мс
POST /predict	504	Inference timeout	3.6
POST /predict-queued	504	Inference timeout	3.3
POST /batch-predict	504	Batch inference timeout	3.3

Модели нужно минимум 23 мс, поэтому при таймауте 1 мс ни один запрос не успевает получить прогноз, и клиент получает 504 через несколько миллисекунд. Записи в БД не создаются. По логу видно, что worker успел забрать задачу из очереди до ее отмены, сформировал из нее пакет и завершил его с ошибкой `Batch inference failed`, то есть модель выполнила работу для запроса, на который уже был отправлен ответ.

4.11 Задания 12, 13 и 14. Подключение внешней LLM

Клиент и обработка ошибок провайдера

В `lifespan` создается один `httpx.AsyncClient(timeout=60)`, который закрывается при остановке приложения. Повторное использование клиента позволяет сохранять пул TCP/TLS-соединений между запросами. Функция `call_vireonix_llm` отправляет `POST {LLM_BASE_URL}/chat/completions` в OpenAI-совместимом формате (system- и user-сообщения, `temperature`, `max_tokens`) и преобразует ошибки в HTTP-коды (таблица 7).

Таблица 7. Ошибки вызова LLM и коды ответа сервиса

Ситуация	Код	Проверено
`httpx.TimeoutException`	504	заглушка отвечает 10 с при таймауте 3 с
`httpx.RequestError` (нет соединения, DNS, отказ прокси)	503	Vireonix недоступен; закрытый порт
провайдер вернул статус, отличный от 200	502	заглушка отвечает 500
невалидный JSON, нет `choices`, пустой `content`	502	по коду

POST /llm/chat

Endpoint принимает `LLMChatRequest`: `prompt` длиной от 1 до 5000 символов (пустая строка дает 422), `model` (по умолчанию `auto`), `temperature` от 0 до 2 и `max_tokens` от 1 до 2000. Ответ `LLMChatResponse` содержит `request_id`, `requested_model`, `returned_model`, `answer`, `finish_reason`, `usage` и `latency_ms`. Поле `latency_ms` показывает время самого вызова провайдера, его можно сравнить с `X-Process-Time` всего запроса.

Скриншот из Swagger UI: сохраните как `img/llm_chat.png`

Рис. 8. POST /llm/chat: запрос «Объясни, что такое агроскоринг»

Если провайдер недоступен по сети, сервис возвращает 503 с `request_id` (рисунок 9). Этот результат соответствует тест-кейсу 15.

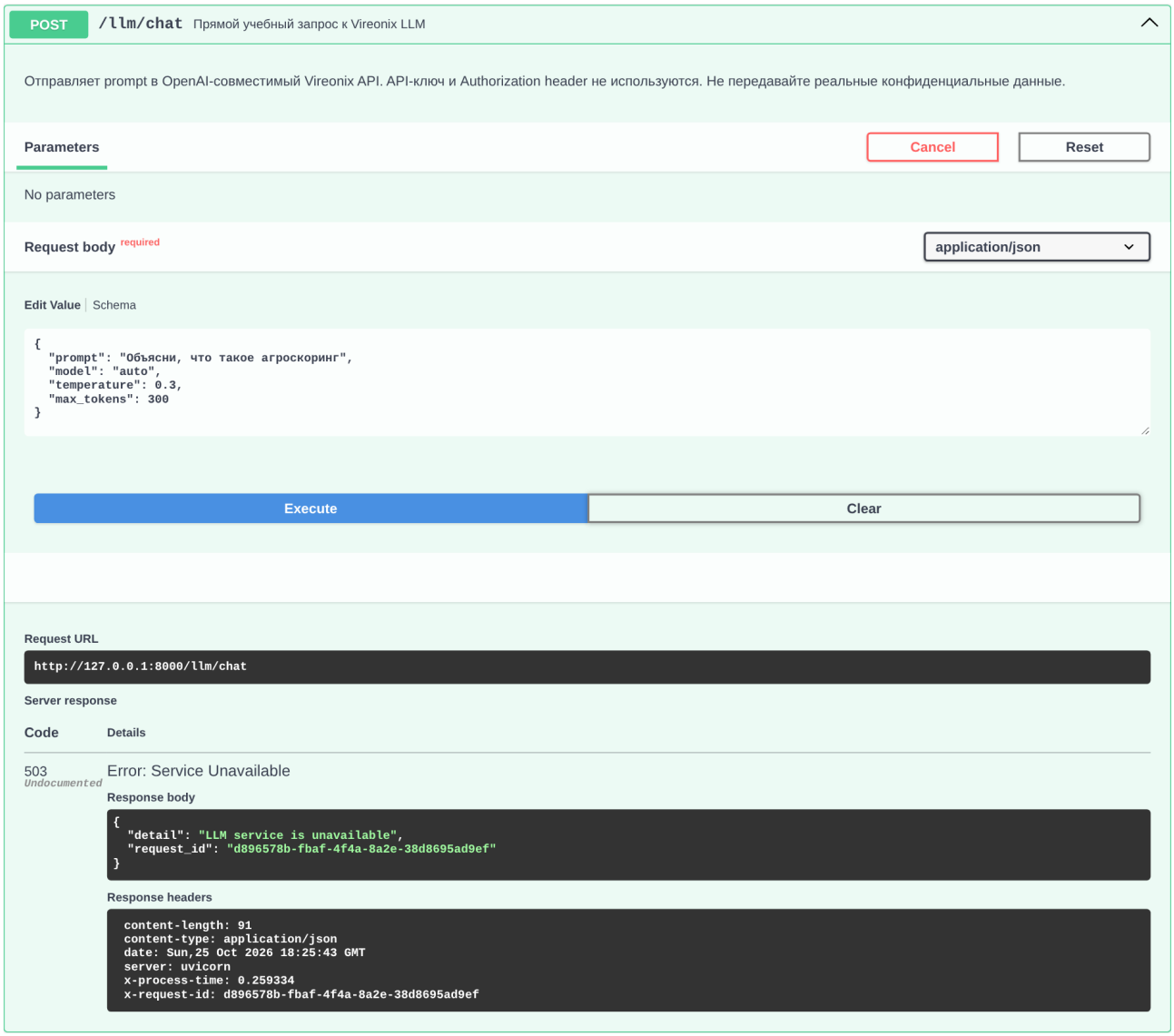


Рис. 9. POST /llm/chat при недоступном провайдере: 503 LLM service is unavailable

POST /predict-with-llm

В этом endpoint расчет риска и текстовое объяснение выполняются раздельно:

1. скоринг выполняется так же, как в `/predict`: валидация, вызов модели с таймаутом, сохранение в БД с `source = "single+llm"`;
2. промпт для LLM формируется функцией `build_scoring_explanation_prompt` только из результата: `risk_score`, `risk_level`, рекомендация и версия модели. Поле `farm_id` и исходные финансовые признаки (долг, просрочка, дефолты) во внешний сервис не передаются;
3. system-промпт и требования в user-промпте запрещают LLM менять категорию риска и добавлять факторы, которых нет во входных данных;
4. ответ состоит из двух частей: `prediction` с результатом модели и `llm` с текстом объяснения.

Поэтому LLM не влияет на `risk_score` и `risk_level`: эти значения рассчитаны и сохранены в БД до обращения к ней.

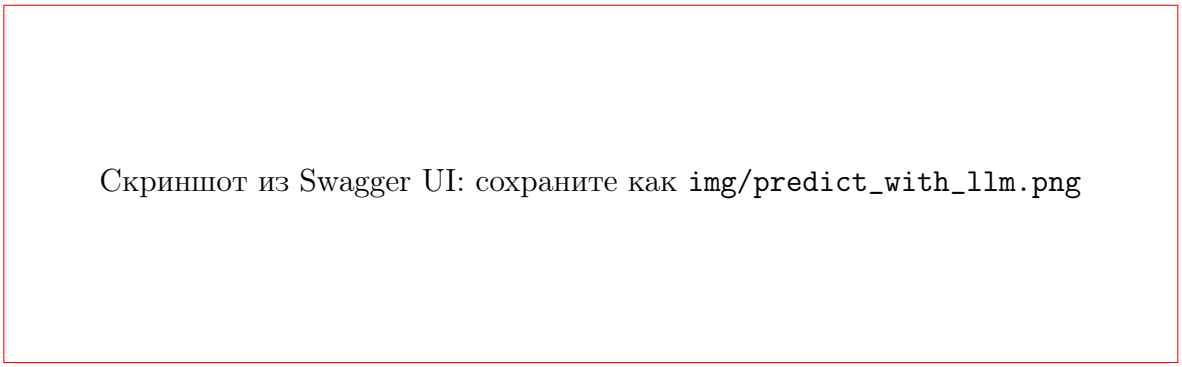


Рис. 10. POST /predict-with-llm: прогноз и текстовое объяснение LLM

Так как прогноз сохраняется до вызова LLM, при сбое LLM клиент получает 502, 503 или 504, а запись в БД остается. Это проверено: запрос `a6675d50-...` к `/predict-with-llm` вернул 503, а `GET /predictions/a6675d50-...` после этого вернул сохраненный прогноз. Найти такую запись можно по `request_id` из тела ответа с ошибкой.

5 Тестирование API (задание 15)

Таблица 8. Тест-кейсы: ожидаемые и полученные HTTP-коды

№	Запрос	Ожидался	Получен	Результат
1	`GET /health`	200	200	пройден
2	корректный `POST /predict`	200	200	пройден
3	`POST /predict`, `area_ha = -100`	422	422	пройден
4	`POST /predict`, `region = "Moscow"`	400	400	пройден
5	`GET /predictions/{id}`, существующий `request_id`	200	200	пройден
6	`GET /predictions/{id}`, неизвестный `request_id`	404	404	пройден
7	корректный `POST /batch-predict` (3 объекта)	200	200	пройден
8	`POST /batch-predict`, 9 объектов	422	422	пройден
9	`POST /predict-queued`	200	200	пройден
10	переполненная очередь: 400 одновременных `/predict-queued`	429	429	пройден (92 из 400)
11	`GET /predictions/by-weekday?weekday=monday&page=1&per_page=5`[1]	200	200	пройден
12	`by-weekday`, `page = -1`	422	422	пройден
13	`by-weekday`, `weekday = funday`	400	400	пройден
14	корректный `POST /llm/chat`	200	?	?
15	недоступный LLM, ошибка провайдера	502/503/504	503, 502, 504	пройден (сеть, ответ 500, таймаут)
16	корректный `POST /predict-with-llm`	200	?	?

6 API как контракт (задание 16)

В таблице 9 приведены все endpoints сервиса. Коды ошибок проверены по коду обработчиков и по фактическим ответам. Код 500 может вернуть любой endpoint при необработанном исключении, поэтому он не указан в таблице.

Таблица 9. API-контракт сервиса

Endpoint	**Method**	**Input**	**Output**	**Success**	**Ошибки**
/health	GET	нет	HealthResponse	200	нет
/model-info	GET	нет	ModelInfoResponse	200	нет
/predict	POST	FarmRequest	PredictionResponse	200	400, 422, 503, 504
/batch-predict	POST	BatchRequest	BatchPrediction-Response	200	400, 422, 503, 504
/predict-queued	POST	FarmRequest	PredictionResponse	200	400, 422, 429, 503, 504
/queue-status	GET	нет	размер очереди и настройки	200	нет
/predictions	GET	limit, risk_level (query)	List[Prediction-Response]	200	400, 422, 503
/predictions/{request_id}	GET	request_id (path)	PredictionResponse	200	404, 409, 503
/requests/{request_id}/predictions	GET	request_id (path)	List[Prediction-Response]	200	404, 503
/predictions/by-weekday	GET	weekday, page, per_page (query)	PaginatedPredictions-Response	200	400, 404, 422, 503
/demo/seed-weekdays	POST	count (query)	статус и число записей	200	422, 503
/llm/chat	POST	LLMChat-Request	LLMChatResponse	200	422, 502, 503, 504
/predict-with-llm	POST	FarmRequest	PredictWithLLM-Response	200	400, 422, 502, 503, 504

По сравнению с таблицей из задания для `/predictions/by-weekday` добавлен код 503 (ошибка чтения БД), также добавлены endpoints чтения по `request_id`.

В OpenAPI-схеме описаны только ответы 200 и 422, поэтому Swagger UI отмечает остальные ответы как *Undocumented* (рисунок 9). Чтобы коды ошибок отображались в документации, их можно описать в параметре `responses` декоратора, например `responses={429: {"model": ErrorResponse}}`.

7 Путь одного запроса через систему

Ниже описан путь запроса `POST /predict-with-llm`, который проходит через все компоненты сервиса.

[1]Параметр `weekday` обязательный, без него этот запрос возвращает 422.

1. Клиент отправляет запрос, uvicorn передает его в middleware. Middleware фиксирует время начала, берет или создает `request_id`, сохраняет его в `request.state` и пишет в лог `Request started`.
2. Pydantic разбирает тело запроса в `FarmRequest`. При ошибке обработчик возвращает 422 с `request_id`, и функция endpoint не вызывается. Затем проверяются `MODEL_READY` (503) и бизнес-правила (400).
3. `model_predict_single` захватывает `model_lock` и выполняется внутри `asyncio.wait_for` с таймаутом 2 с (504). В `/predict-queued` на этом шаге запрос проходит через очередь: `put_nowait` (429 при переполнении), ожидание пакета до 50 мс, общий вызов модели и получение результата через `Future`.
4. Функции `get_risk_level` и `get_recommendation` преобразуют score в уровень риска и рекомендацию.
5. `save_prediction` открывает соединение с БД, выполняет параметризованный `INSERT` и `COMMIT`. При ошибке возвращается 503, и клиент не получает 200 для несохраненного результата.
6. Из результата скоринга (без исходных данных хозяйства) формируется промпт, `httpx.AsyncClient` асинхронно отправляет его провайдеру. Пока ответ не получен, цикл событий обрабатывает другие запросы. Ошибки провайдера возвращаются как 502, 503 или 504.
7. FastAPI сериализует `PredictWithLLMResponse`, middleware добавляет заголовки `X-Request-ID` и `X-Process-Time` и пишет в лог `Request completed`.

Распределение времени видно по логу. Для `/predict-queued`: 1 мс до постановки в очередь, 50 мс ожидания пакета, 23 мс работы модели и около 5 мс записи в БД, всего 79.6 мс. Запрос `/predict` без очереди и LLM занимает около 29 мс, из них 23 мс приходится на модель. В `/predict-with-llm` к этому добавляется время ответа LLM. Этот этап самый долгий и чаще других может завершиться ошибкой, поэтому скоринг и сохранение выполняются до него и от его результата не зависят.

8 Выводы

В ходе работы API агроскоринга дополнен механизмами, которые нужны для эксплуатации AI-сервиса. Все задания выполнены и проверены тест-кейсами из раздела 5 и сценариями отказов из раздела 4.2.

- `request_id` связывает ответ клиенту, заголовки, записи лога и строки БД. Клиент может передать свой идентификатор, тогда запрос можно отследить в нескольких системах. Идентификатор в теле ответа с ошибкой позволяет найти этот сбой в логах.
- Batching увеличивает пропускную способность, так как фиксированные затраты на вызов модели приходятся на весь пакет. На 64 объектах dynamic batching увеличил производительность с 40 до 88 объектов/с, явный batching до 164 объектов/с. При этом растет задержка одиночного запроса в очереди (окно 50 мс), поэтому размер пакета и время ожидания нужно подбирать с учетом требований к latency и throughput.
- Backpressure ограничивает рост памяти и задержек при перегрузке. Лишние запросы сразу получают 429 с `Retry-After`, а принятые укладываются в таймаут: из 400 одновременных запросов 308 обработаны успешно, 92 отклонены, таймаутов не было.

- База данных обеспечивает постоянное хранение результатов, поиск по `request_id` и постраничный просмотр. Асинхронный драйвер не блокирует цикл событий, а запись пакета одной транзакцией атомарна и примерно в 8 раз быстрее отдельных вызовов.
- LLM используется только для текстового объяснения результата. Значение риска рассчитывает детерминированная модель, в LLM передается только итог скоринга без персональных и финансовых данных, а ошибки LLM возвращаются клиенту кодами 502, 503 и 504.